

RideSmart: Modelagem e Análise de Rotas Urbanas com Grafos

Alan Cesar Reboucas de Araujo Carvalho
Erick Henrique da Silva Paz
Matheus Silva Mendes
Algoritmos e Estruturas de Dados II
Universidade Federal do Rio Grande do Norte
Natal, Brasil

Resumo—Este trabalho modela o problema de escolha de ponto de embarque em mobilidade urbana como um grafo de rede viária real. Dado um ponto de origem A , um destino B e uma distância máxima de caminhada X , busca-se o ponto P que minimiza o custo total da rota $A \rightarrow P$ (a pé) + $P \rightarrow B$ (carro). Quatro algoritmos de caminho mínimo foram implementados e comparados: Dijkstra Simples, Dijkstra com Heap, A^* com heurística geográfica e Dijkstra Bidirecional. Os experimentos foram realizados sobre a malha viária real de Natal/RN, extraída do OpenStreetMap via OSMnx.

I. INTRODUÇÃO

Aplicativos de mobilidade urbana frequentemente orientam o usuário a caminhar até um ponto de embarque próximo, em vez de aguardar o veículo na porta. Esse deslocamento a pé pode reduzir a distância total de carro, especialmente em áreas com vias de mão única ou acesso restrito.

Este trabalho tem como objetivos: (i) modelar o problema como grafo; (ii) implementar e comparar algoritmos de caminho mínimo; (iii) analisar diferentes critérios de custo (distância, tempo e tempo com trânsito sintético); e (iv) discutir o trade-off entre caminhar mais e chegar mais rápido.

O cenário utilizado tem como origem (A) o Museu de Morfologia da UFRN e como destino (B) o Assaí Atacadista, ambos em Natal/RN, com distância máxima de caminhada $X = 500$ m.

II. MODELAGEM DO PROBLEMA

A. Representação como grafo

A rede viária foi obtida via OSMnx com dados do OpenStreetMap. Foram utilizados dois grafos distintos, ambos do tipo multigrafo direcionado (MultiDiGraph):

- **Grafo drive** (raio 4 km): usado no trecho $P \rightarrow B$ de carro. Contém 4.963 nós e 12.356 arestas.
- **Grafo walk** (raio 700 m ao redor de A): usado para encontrar candidatos P e calcular a caminhada $A \rightarrow P$. Contém 925 nós e 2.584 arestas.

Cada **nó** representa uma interseção viária; cada **aresta** representa um segmento de rua com atributos de comprimento, velocidade e tempo de viagem.

B. Pesos utilizados

Três funções de custo foram modeladas:

- `length` (m): distância física da aresta.
- `travel_time` (s): tempo estimado pelo OSMnx sem trânsito.
- `travel_time_transito` (s): tempo com fator de congestionamento sintético, gerado multiplicando `travel_time` por um fator aleatório $U[1,0; 3,5]$ por aresta (`seed=42`).

A velocidade de caminhada foi fixada em $v_{\text{walk}} = 1,4$ m/s.

C. Definição dos cenários

O ponto P é qualquer nó da rede pedestre a no máximo $X = 500$ m de A (82 candidatos encontrados). A é excluído dos candidatos; o caso sem caminhada é tratado como cenário 4. Cinco cenários foram comparados:

- 1) **Menor distância**: P que minimiza $\text{distância}(A \rightarrow P) + \text{distância}(P \rightarrow B)$.
- 2) **Mais rápido s/ trânsito**: P que minimiza tempo total usando `travel_time`.
- 3) **Mais rápido c/ trânsito**: idem usando `travel_time_transito`.
- 4) **Sem caminhada**: rota $A \rightarrow B$ inteiramente de carro (referência).
- 5) **Ganho ao caminhar**: diferença de custo entre cenário 4 e os cenários 1/2.

III. ALGORITMOS IMPLEMENTADOS

A. Dijkstra Simples — $O(V^2)$

Implementado manualmente. A cada iteração, percorre linearmente todos os nós não visitados para selecionar o de menor distância acumulada ($O(V)$ por iteração). Serve como referência didática para comparação com a versão com heap.

B. Dijkstra com Heap — $O((V + E) \log V)$

Implementado manualmente com `heapq`. Extrai o nó de menor custo em $O(\log V)$, reduzindo drasticamente o número de operações em grafos esparsos. Usado como algoritmo principal para avaliar todos os 82 candidatos P .

C. A* com heurística geográfica

Extensão do Dijkstra com função heurística $h(u)$ baseada na distância Haversine entre o nó atual e B. Para `length`, $h(u)$ é a distância em metros (admissível). Para `travel_time`, converte-se pela velocidade máxima do grafo (120 km/h $\approx$ 33,3 m/s). Complexidade teórica $O((V+E) \log V)$, com menor constante quando a heurística é eficaz.

D. Dijkstra Bidirecional (literatura)

Executa duas buscas simultâneas: da origem A e do destino B no grafo reverso. As frentes convergem pelo meio do caminho, reduzindo o número de nós expandidos. Implementado via `nx.bidirectional_dijkstra` do NetworkX. Mantém complexidade $O((V+E) \log V)$ com melhor desempenho prático para pares distantes.

IV. EXPERIMENTOS E RESULTADOS

A. Validação

Antes dos experimentos reais, ambas as implementações manuais foram validadas em um grafo didático com quatro nós e duas rotas entre A e B: rota via X (600 m, 240 s) e rota via Y (1.000 m, 120 s). Dijkstra Simples e Heap encontraram corretamente a rota via X ao minimizar distância e via Y ao minimizar tempo, confirmando equivalência dos resultados.

B. Comparação dos 5 cenários

A Tabela I apresenta os resultados para o Dijkstra Heap.

Tabela I
RESULTADOS DOS 5 CENÁRIOS — DIJKSTRA HEAP

Cenário	Dist. (m)	Tempo (s)
1 – Menor dist. (P, walk 458 m)	2.990,1	533,8
2 – Mais rápido s/ trânsito (P, walk 144 m)	3.238,5	320,9
3 – Mais rápido c/ trânsito (P, walk 144 m)	3.238,5	630,9
4 – Sem caminhada (A→B direto)	3.238,5	320,9
5 – Ganho ao caminhar (vs. C4)	+248,3	0,0

A menor distância foi também a mais rápida? Não. O cenário 1 reduz em 248 m a distância total (ganho real), mas leva 533,8 s contra 320,9 s do cenário 4. O P escolhido em C1 está sobre a própria rota direta A→B, portanto caminhar até ele apenas substitui um trecho inicial de carro por caminhada — a distância cai, mas o tempo piora por causa da menor velocidade a pé.

Caminhar valeu a pena? Em distância, sim (248 m economizados no cenário 1). Em tempo, não houve ganho. O cenário 4 (sem caminhada) apresentou uma limitação importante da modelagem: quando o ponto de origem A não possui nó correspondente no grafo de carro, o algoritmo é obrigado a percorrer um trecho a pé (144 m) para alcançar a rede viária — tornando o cenário 4 equivalente ao cenário 2 em tempo (320,9 s) e distância (3.238,5 m). Na prática, não foi possível partir inteiramente de carro a partir de A.

Caminhar atrapalhou? Sim, no cenário 1: o P de menor distância exige 458 m a pé por ruas mais lentas de carro, tornando a rota 66% mais lenta que o cenário 4 (533,8 s vs. 320,9 s). Caminhar mais reduziu a distância mas piorou o tempo.

Efeito do trânsito sintético: O cenário 3 manteve o mesmo P do cenário 2, mas o tempo subiu de 320,9 s para 630,9 s (+96,6%), mostrando o impacto do fator de congestionamento (médio de $2,25\times$ por aresta). O trânsito não alterou a escolha do P, mas alterou substancialmente o custo estimado.

C. Dijkstra Simples vs. Heap

Tabela II
BENCHMARK SIMPLES VS. HEAP — EXECUÇÃO ÚNICA A→B

Peso	Simples (s)	Heap (s)	Speedup
Distância	0,6875	0,0105	65×
Tempo	1,0088	0,0176	57×

O Heap foi 57–65× mais rápido em uma única execução. Ao avaliar os 82 candidatos P, o Heap concluiu em 2,62 s e o Simples em 135,67 s (51,7× mais lento). Ambos produziram resultados idênticos em todos os cenários, confirmando equivalência matemática.

D. A* e Dijkstra Bidirecional

O A* apresentou desempenho inferior ao Heap neste experimento: 3–8× mais lento, dependendo do peso. O A* guia a busca com uma heurística geográfica (distância Haversine até B), expandindo apenas nós na direção do destino e ignorando regiões do grafo improváveis para o caminho ótimo. Isso resulta em menos nós expandidos em relação ao Dijkstra, embora neste experimento o overhead do cálculo da heurística por nó tenha superado esse benefício. Os resultados foram idênticos ao Heap em todos os cenários.

O Dijkstra Bidirecional trouxe ganho real de desempenho: foi 1,88× (distância) a 3,5× (tempo) mais rápido que o Heap unidirecional, com resultados de distancia parecidos com o Heap, porém o Bidirecional conseguiu encontrar pontos de embarque mais proximos. O Dijkstra Bidirecional trouxe ganho qualitativo em relação ao Dijkstra convencional: ao executar duas buscas simultâneas (de A e de B no grafo reverso), reduz o espaço de busca expandindo nós apenas até o ponto de convergência no meio do caminho, em vez de explorar todo o grafo a partir de uma única origem. Isso é especialmente vantajoso para pares A–B distantes, como no cenário estudado. Os resultados foram idênticos aos demais algoritmos em todos os cenários.

V. DISCUSSÃO

Limitações da modelagem: (i) O trânsito sintético é aleatório e uniforme por aresta, sem considerar horário ou tipo de via. (ii) A velocidade de caminhada é constante, ignorando relevo, semáforos e calçadas. (iii) O critério de menor distância pode selecionar um P sobre a rota direta, sem benefício real de

tempo. (iv) A heurística do A* pode ser prejudicial em grafos densos quando seu custo de cálculo supera o ganho em nós expandidos. (v) Nós do grafo walk podem não existir no grafo drive, exigindo filtro de compatibilidade.

Aproximação com aplicativo real: Para aproximar o modelo de um sistema real de mobilidade, seria necessário: substituir o trânsito sintético por dados históricos ou em tempo real; modelar o tempo de espera pelo veículo em cada candidato P, que varia com a oferta de motoristas na região; combinar múltiplos critérios simultaneamente (tempo, distância e custo monetário); personalizar a velocidade de caminhada por perfil de usuário; e garantir que origem e destino estejam sempre mapeados sobre a rede viária de carro, evitando a ambiguidade observada no cenário 4.

VI. CONCLUSÃO

Este trabalho modelou o problema de escolha de ponto de embarque em mobilidade urbana como caminho mínimo em grafo real, com dois modos de transporte distintos (drive e walk). Os resultados mostram que caminhar até um P alternativo pode reduzir a distância total (248 m no cenário estudado), mas não necessariamente o tempo de viagem. Uma limitação prática relevante foi que o ponto de origem A não possuía nó no grafo de carro, obrigando o algoritmo a percorrer 144 m a pé mesmo no cenário sem caminhada — o que aproximou os cenários 2 e 4 e dificultou a comparação direta entre caminhar e não caminhar.

Entre os algoritmos, o Dijkstra Heap apresentou o melhor custo-benefício (57–65× mais rápido que o Simples, com resultados idênticos). O A* expandiu menos nós ao guiar a busca com heurística geográfica, e o Bidirecional reduziu o espaço de busca ao convergir pelo meio do caminho — ambos encontrando os mesmos resultados ótimos. Como trabalhos futuros, sugere-se incorporar dados reais de tráfego, modelar o tempo de espera pelo veículo em cada P e garantir que os pontos de origem e destino estejam sobre a rede viária de carro.